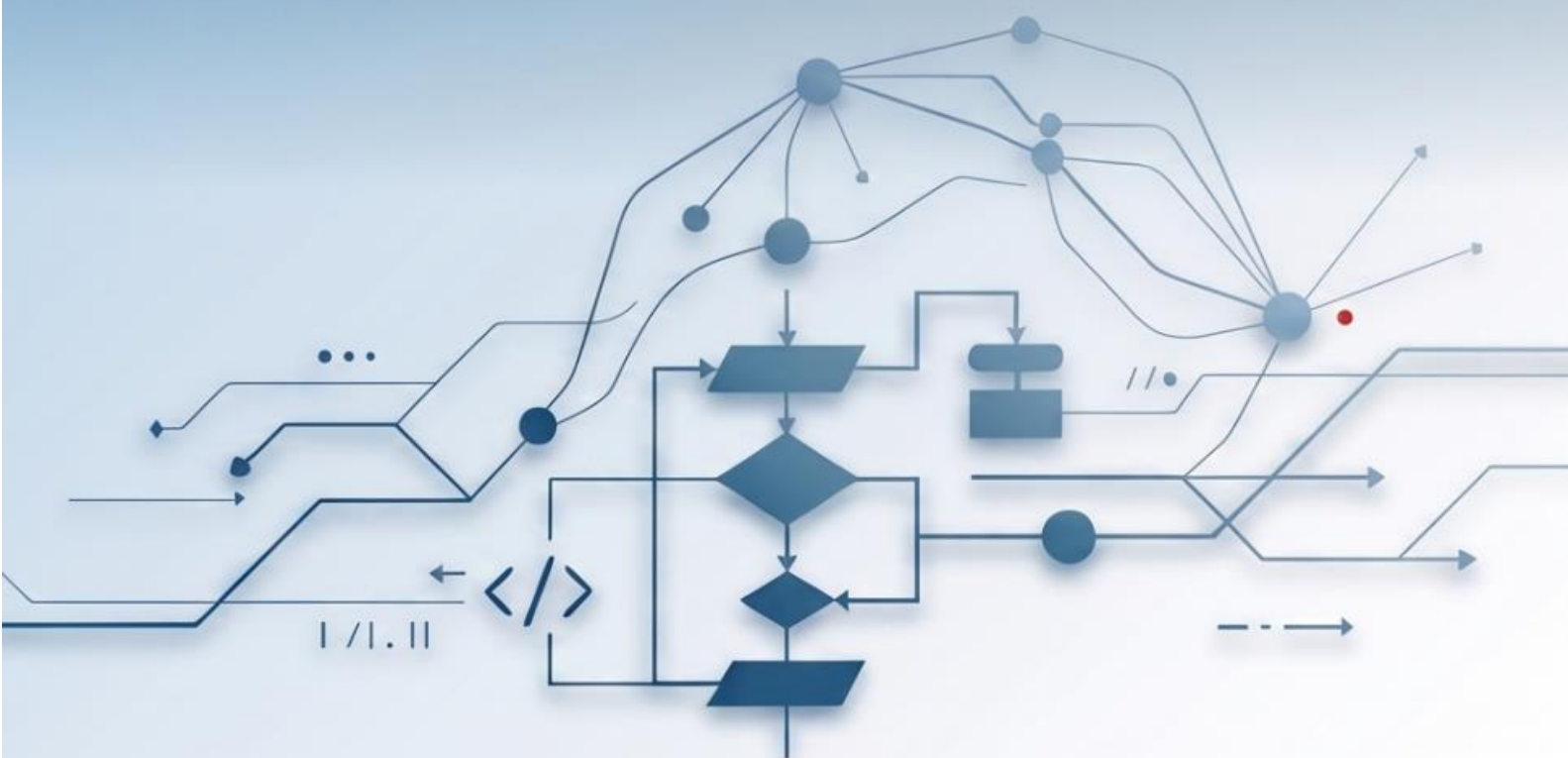


UNIVERSIDAD DE OVIEDO
ESCUELA DE INGENIERÍA INFORMÁTICA DE OVIEDO (EII)
FACULTAD DE CIENCIAS
SEGUIMIENTO DE PRÁCTICAS

ALGORITMIA - SEGUIMIENTO DE PRÁCTICAS

CURSO ACADÉMICO 2025-2026



ESTUDIANTE: NICOLÁS PÉREZ

GRUPO DE PRÁCTICAS: PL-2

UO: 306041

FECHA DE ENTREGA: 1 DE MAYO DE 2026

ÍNDICE

PRACTICA P0	
Factor 1	
Factor 2	
PRACTICA P11	
Toma de tiempos de ejecución	
Crecimiento del tamaño del problema	
Toma de tiempos pequeños (<50ms)	
PRACTICA P12	
PRACTICA P2	
EJERCICIO 1: algoritmo burbuja	
EJERCICIO 2: algoritmo selección	
EJERCICIO 3: algoritmo inserción	
EJERCICIO 4: algoritmo rápido	
EJERCICIO 5: algoritmo rápidoBajos vs inserciónBajos	
EJERCICIO OPCIONAL	
PRACTICA P3	
DyV por SUSTRACCIÓN	
DyV por división:	
Dos ejemplos básicos:	
Practica DyV	
Práctica P4	
IMPLEMENTACIÓN EN JAVA	
IMPLEMENTACIÓN EN PYTHON	
Práctica P5	
Descripción del problema	
Definición del estado	
Relación de recurrencia	
Complejidad	
Implementación	
Conclusión	
Práctica P6	
TIEMPOS DEL ALGORITMO DE LOS CONTENEDORES	
Complejidad del algoritmo	
Práctica P7	
Descripción del problema y punto de partida	
Heurístico mejorado: poda por cota inferior	
Diferencias respecto a P6	
Complejidad y análisis práctico	
Conclusión	

PRACTICA P0

Factor 1

En este apartado se analiza cómo influye la potencia del hardware en el rendimiento de un programa. Para ello, se ha ejecutado exactamente el mismo código fuente (PythonA1.py) en dos ordenadores distintos (PC 1 y PC 2) sometiéndolos a las mismas cargas de trabajo (mismos valores de n).

El objetivo es comparar los tiempos de ejecución resultantes y demostrar cómo las características técnicas de cada equipo (principalmente la CPU) afectan a la velocidad de resolución del problema.

Especificaciones de los equipos:

Ordenador 1 (PC_1 - Referencia):

- **Procesador:** 12th Gen Intel(R) Core(TM) i5-12400 @ 2.50 GHz
- **Memoria RAM:** 16,0 GB

Ordenador 2 (PC_2 - Alto Rendimiento):

- **Procesador:** AMD Ryzen 7 7800X3D 8-Core Processor @ 4.20 GHz
- **Memoria RAM:** 32,0 GB

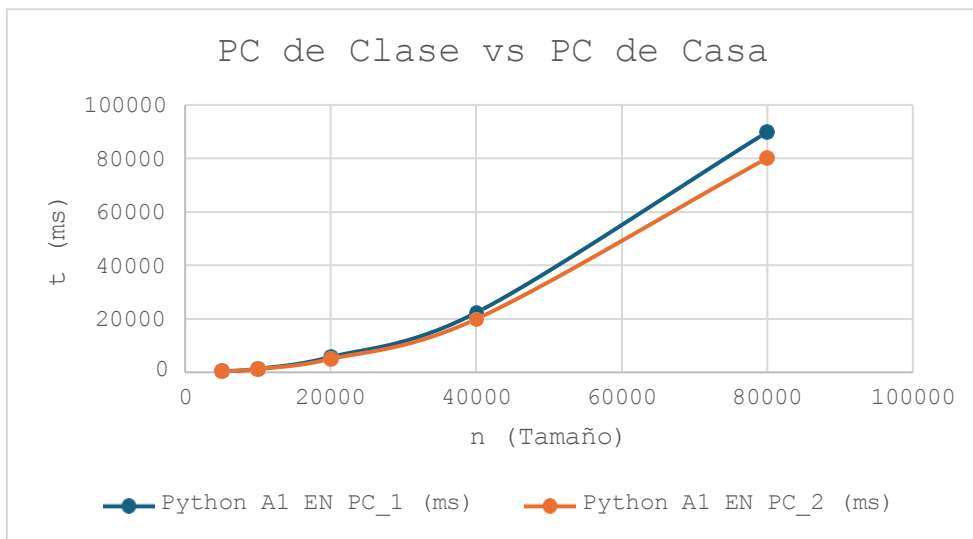
A priori, se espera que el PC_2 obtenga mejores tiempos debido a su mayor frecuencia base (4.20 GHz frente a 2.50 GHz) y a que se trata de un procesador de gama entusiasta frente a la gama media del i5.

Resultados Experimentales

A continuación, se presentan los tiempos obtenidos en milisegundos para ambos equipos:

n (Tamaño)	Python A1 EN PC_1 (ms)	Python A1 EN PC_2 (ms)
5000	335	334
10000	1347	1242
20000	5738	4999
40000	22318	19928
80000	89896	80168

A continuación, se adjunta también una gráfica comparativa:



Conclusión Factor 1

Se demuestra que un hardware más potente (mayor frecuencia de CPU y mejor arquitectura) reduce el tiempo de ejecución de manera constante. Sin embargo, dado que el Algoritmo A1 es ineficiente por naturaleza (complejidad cuadrática).

Factor 2

En este apartado se aborda el factor más determinante en el rendimiento: el diseño del algoritmo. Mientras que el hardware o el lenguaje pueden mejorar el tiempo en un factor constante (x2, x10), un buen algoritmo puede reducir la complejidad teórica del problema, logrando mejoras de varios órdenes de magnitud.

Se han implementado y medido tres evoluciones sobre el algoritmo base:

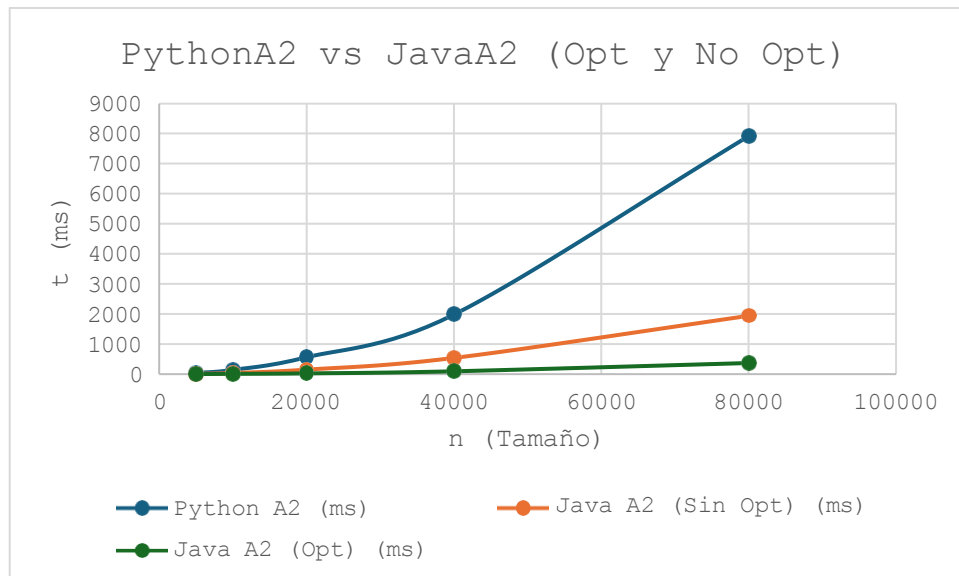
- **Algoritmo A2 (Rotura de bucle):** Mejora respecto a A1 deteniendo la búsqueda en cuanto se encuentra el primer divisor.
- **Algoritmo A3 (Raíz Cuadrada):** Optimización matemática que reduce el espacio de búsqueda. Solo se comprueban divisores hasta $\sqrt{n}$ y se omiten los números pares.
- **Algoritmo A4 (Criba de Eratóstenes):** Cambio de paradigma. En lugar de divisiones por tentativa, utiliza un array de booleanos para "tachar" múltiplos. Cambia tiempo de CPU por consumo de memoria.

Resultados Experimentales

La siguiente tabla muestra los tiempos obtenidos para diferentes tamaños de n tanto en Python como en Java (con y sin optimización JIT):

n (Tamaño)	Python A2 (ms)	Java A2 (Sin Opt) (ms)	Java A2 (Opt) (ms)
5000	37	10	3
10000	141	40	10
20000	561	150	25
40000	1983	540	95
80000	7920	1940	370

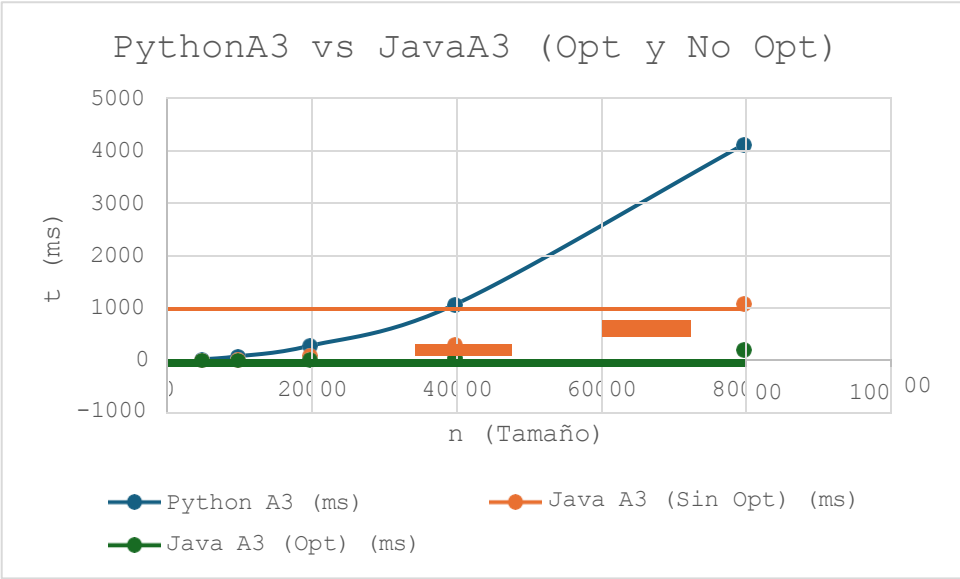
Con su respectiva gráfica:



Ahora el algoritmo A3:

Python A3 (ms)	Java A3 (Sin Opt) (ms)	Java A3 (Opt) (ms)
20	6	0
79	16	0
280	84	17
1064	300	50
4115	1075	192

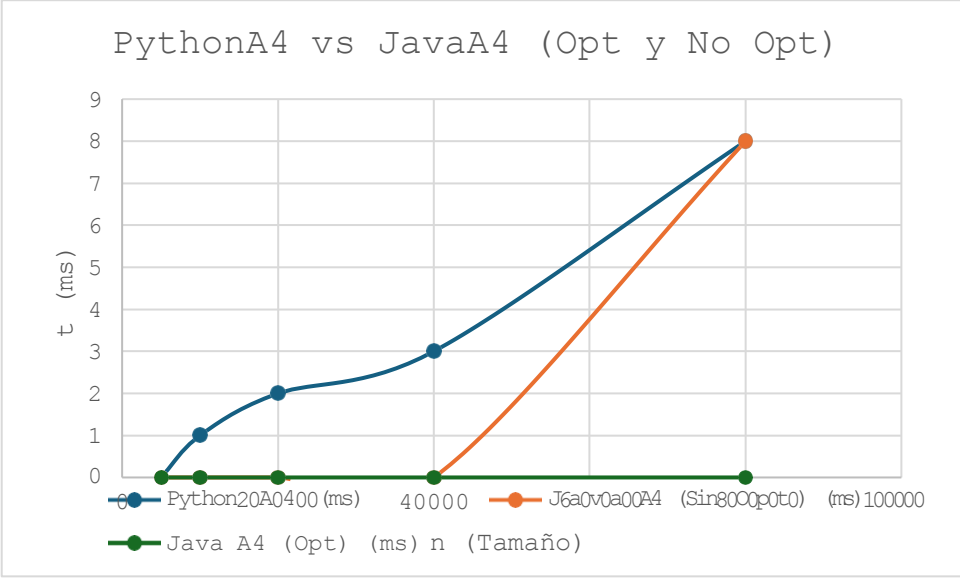
Con su gráfica:



Y, por último, el algoritmo A4 y el más eficiente:

Python A4 (ms)	Java A4 (Sin Opt) (ms)	Java A4 (Opt) (ms)
0	0	0
1	0	0
2	0	0
3	0	0
8	8	0

Con su gráfica:



Diferencia entre Java y Python

Los resultados demuestran claramente que Java es mucho más rápido que Python para este tipo de cálculos. La razón es sencilla:

- **Python (Interpretado):** Funciona como un traductor simultáneo. Lee el código línea por línea y lo traduce al ordenador mientras se ejecuta. Esto hace que pierda tiempo en cada vuelta del bucle "pensando" qué tiene que hacer.
- **Java (Compilado/JIT):** Funciona como si leyera un libro ya traducido. Gracias a su compilador (JIT), transforma el código a un lenguaje que el procesador entiende directamente. Una vez que "aprende" lo que hace el bucle, lo ejecuta a toda velocidad sin detenerse a traducir.

Conclusión

Este experimento valida la premisa fundamental de la asignatura: **El diseño del algoritmo es el factor dominante.**

Mientras que cambiar del PC 1 al PC 2 (Factor 2) mejoraba el tiempo en un 10%, cambiar del Algoritmo A1 al Algoritmo A4 reduce el tiempo en un factor de 10.000 (de casi 90 segundos a 8 milisegundos en Python). Ninguna mejora de hardware puede competir con una reducción en la complejidad algorítmica.

PRACTICA P11

Toma de tiempos de ejecución Crecimiento del tamaño del problema Toma de tiempos pequeños (<50ms)

Pregunta 1: Calcula, ¿cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo:

El método que se utiliza devuelve un número de tipo long, por tanto, este es un entero de 64 bits. El valor más grande que almacena un long $2^{63} - 1$ lo cual equivale a 9.223.372.036.854.775.807 milisegundos.

Teniendo en cuenta que partimos desde 1970 aproximadamente le quedan 292 millones de años. En ese momento, el sistema colapsará.

Pregunta 2: ¿Por qué a veces el tiempo medido sale 0? ¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables?:

Esto se debe a que la medición se hace en ms y los procesadores actuales pueden realizar cuentas como la suma muy rápido, en el orden incluso de los nanosegundos. Además, estamos trabajando con enteros long, por lo que si un algoritmo tarda 0,0005 ms en ejecutarse Java lo redondeará a 0.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector2 10000000
t1=1770759563056 *** t2=1770759563144
n= 10000000 Tiempo metodo suma = 88
Resultado de la suma de elementos = -292913
```

Según lo medido aproximadamente en $n = 10000000$ el valor ya supera los 50 ms, en concreto, 88 ms.

Mediciones de Vector4.java:

Con parámetro 1:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector4 1
repeticiones = 1
n      Tiempo
10000  0
20000  0
40000  0
80000  0
160000 0
320000 0
640000 7
1280000 10
2560000 22
5120000 33
10240000 55
20480000 106
40960000 201
81920000 411
Fin de la medicion de tiempos *****
```

Con parámetro 10:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector4 10
repeticiones = 10
n      Tiempo
10000  0
20000  0
40000  0
80000  3
160000 14
320000 16
640000 55
1280000 104
2560000 126
5120000 360
10240000 566
20480000 990
40960000 1959
81920000 3975
Fin de la medicion de tiempos *****
```

Con parámetro 100:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector4 100
repeticiones = 100
n      Tiempo
10000  7
20000  10
40000  28
80000  62
160000 116
320000 176
640000 346
1280000 650
2560000 1332
5120000 2550
10240000 4920
20480000 10245
40960000 19548
81920000 38588
Fin de la medicion de tiempos *****
```

Hasta aquí el tiempo era razonable, a partir de aquí la medición se dispara, como podemos ver con 1000 o 10000.

Con parámetro 1000:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector4 1000
repeticiones = 1000
n      Tiempo
10000  91
20000  168
40000  343
80000  471
160000 1306
320000 1992
640000 4716
1280000 7430
2560000 15230
5120000 29434
10240000 58255
20480000 115146
40960000 221243
81920000 415823

Fin de la medicion de tiempos *****
```

Y por último 10000, en la que el tiempo alcanza la hora y media en el último paso:

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041>java -Xint p11.Vector4 10000
repeticiones = 10000
n      Tiempo
10000  576
20000  1133
40000  2180
80000  4366
160000 8588
320000 17202
640000 34870
1280000 71006
2560000 140100
5120000 281891
10240000 568375
20480000 1105234
40960000 2217148
81920000 4564232

Fin de la medicion de tiempos *****
```

Pregunta 3: ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?:

Depende directamente de la complejidad del algoritmo, por ejemplo, en Tsuma, Tmaximo y Tcoincidencias2 ($O(n)$) el tiempo se duplica. Por otro lado, en Tcoincidencias1 ($O(n^2)$) el tiempo se cuadruplica.

Pregunta 4: ¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para $k=3$ y $k=4$ y compruebe los tiempos obtenidos.)

Pues de manera similar al caso anterior, en caso de $O(n)$ el tiempo sería 3 ($k=3$) o 4 veces mayor ($k=4$) y en el caso de $O(n^2)$ el tiempo sería inclusive 9 o 16 veces mayor.

Pregunta 5: ¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal $O(n)$?

En un algoritmo de complejidad $O(n)$, el tiempo de ejecución sigue la función $T(n) = c \cdot n$ (donde c es una constante que depende de la velocidad del procesador y del flag).

Tras programar Vector5.java, Vector6.java y Vector7.java hemos obtenido las siguientes gráficas.

Primero con la comparación de **Tsuma(250)** y **Tmaximo(250)**:

n	T_{suma}	T_{maximo}
10000	16	21
20000	39	37
40000	60	74
80000	147	150
160000	250	270
320000	495	533
640000	892	1069
1280000	1850	2156
2560000	3830	4299
5120000	7347	8740
10240000	14574	17289
20480000	28653	34558
40960000	56621	69259
81920000	103941	141083

En este caso podemos ver que los tiempos con el parámetro 250 son bastante similares, ya que ambas son $O(n)$ al ser la T_{maximo} más cara computacionalmente que T_{suma} se debe la diferencia leve de tiempo, pero no es nada sustancial. En cambio, se espera una diferencia mayor en $O(n^2)$.

A continuación, vemos la diferencia **Tcomparacion1(1)** y **Tcomparacion2(1)**:

n ($fdT > 5min$)	$T_{coincidencias1}$	$T_{coincidencias2}$
10000	584	0
20000	2222	0
40000	9114	0
80000	35325	0
160000	140751	0
320000	561340	0
640000	fdT	15
1280000	fdT	16
2560000	fdT	32
5120000	fdT	46
10240000	fdT	95
20480000	fdT	189
40960000	fdT	331
81920000	fdT	679

En el caso anterior vemos que la diferencia es mucho mayor debido a la diferencia de complejidad del algoritmo ya que $T_{coincidencias2}$ es $O(n)$ y $T_{coincidencias1}$ es $O(n^2)$. Para dar un poco más de margen se establece de umbral 5 minutos, pero vemos que ni siquiera llega a la mitad y ya sobrepasa este umbral con solo 1 de parámetro, de hecho, si se hacen cálculos se ve que tardaría en acabar el último n más de 500 años.

Indique las características principales (procesador, memoria) del ordenador donde se han medido tiempos.

Características de mi PC:

- Procesador: AMD Ryzen 7 7800X3D 8-Core Processor @ 4.20 GHz
- Memoria RAM: 32,0 GB

PRACTICA P12

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	$TBucle1$	$TBucle2$	$TBucle3$	$TBucle4$
100	0	0	1	0
200	0	0	4	12
400	0	3	15	45
800	0	16	65	379
1600	0	51	267	2803
3200	0	229	1117	22497
6400	1	933	4723	179185
12800	1	4154	20201	Fdt
25600	3	18922	82773	Fdt
51200	7	73948	348930	Fdt

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los cuatro casos.

SE LE PIDE: Implementar tres nuevas clases Bucle5, Bucle6 y Bucle7, que simulen algoritmos iterativos con una complejidad $O(n^2 \log n^2)$, $O(n^3 \log n)$ y $O(n^4)$ respectivamente. Tras implementar las clases, mida sus tiempos de ejecución y rellene la tabla:

Hacemos que partan desde 5:

n	$TBucle5$	$TBucle6$	$TBucle7$
5	1	0	0
10	0	0	0
20	1	0	0
40	0	1	0
80	0	4	12
160	3	25	134
320	16	205	1867

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal de los tres casos.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	T_{Bucle1}	T_{Bucle2}	$t1/t2$
100	0	0	0
200	0	0	0
400	0	3	0
800	0	16	0
1600	0	51	0
3200	0	229	0
6400	1	933	0,001071811
12800	1	4154	0,000240732
25600	3	18922	0,000158546
51200	7	73948	9,46611E-05

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	T_{Bucle3}	T_{Bucle2}	$t3/t2$
100	1	0	0
200	4	0	0
400	15	3	5
800	65	16	4,0625
1600	267	51	5,235294118
3200	1117	229	4,877729258
6400	4723	933	5,062165059
12800	20201	4154	4,863023592
25600	82773	18922	4,374431878
51200	348930	73948	4,718586033

CASO3: MISMO ALGORITMO EN ENTORNOS DE DESARROLLO y/o EJECUCIÓN DIFERENTES

Para finalizar, vamos a implementar el mismo algoritmo en dos entornos diferentes con el objetivo de comparar, para ese algoritmo, dichos entornos (Bucle4.py y Bucle4.java). Posteriormente, mediremos los tiempos en Python y Java SIN OPTIMIZACIÓN. El cociente de tiempos para los mismos tamaños del problema oscilará alrededor de un valor (constante que relaciona ambos entornos). Si esta constante es menor a 1, es tanto mejor el que hemos puesto en el numerador y si es mayor que 1, es tanto mejor el que hemos puesto en el denominador

SE LE PIDE: Tras medir los tiempos, rellenar la tabla:

n	<i>Bucle4.py</i>	<i>Bucle4.java</i>	T_{py}/T_{java}
100	3	0	0
200	22	12	1,833333333
400	177	45	3,933333333
800	1473	379	3,886543536
1600	12601	2803	4,495540492
3200	133328	22497	5,926479086
6400	1136672	179185	6,343566705

PRACTICA P2

EJERCICIO 1: algoritmo burbuja

n	$t_{ordenado}$	$t_{inverso}$	$t_{aleatorio}$
10000	423	1096	889
20000	1680	4374	3430
40000	6675	17418	13867
80000	26847	70587	55167
160000	107794	279205	220076

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo es $O(n^2)$, al duplicar el tamaño del vector de entrada ($2n$) el número de operación debe multiplicarse por $2^2 = 4$. Por tanto, cuadra ya que siempre el valor siguiente esta en torno a 4 veces el anterior.

En cuanto a los diferentes tiempos para un mismo valor de, también concuerdan con lo esperado. Aunque el algoritmo realiza las mismas comparaciones, el número de intercambios en memoria varía drásticamente. Por tanto, cuadra perfectamente que el caso ordenado sea el más rápido (no hace intercambios), el caso inverso sea el más lento (hace el máximo de intercambios) y el caso aleatorio se quede en un tiempo intermedio.

EJERCICIO 2: algoritmo selección

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de Selección:

n	t ordenado	t inverso	t aleatorio
10000	401	350	394
20000	1563	1418	1544
40000	6239	5634	6162
80000	24807	22575	24683
160000	100064	86102	99005

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo de Selección es $O(n^2)$. Al duplicar el tamaño del vector de entrada (), el número de operaciones debe multiplicarse por $2^2 = 4$. Por tanto, cuadra, ya que al dividir el tiempo de un valor de entre el anterior, el resultado está siempre en torno a 4.

En cuanto a los diferentes tiempos para un mismo valor de, también concuerdan con lo esperado. El algoritmo de Selección siempre realiza el mismo número de comparaciones sin importar cómo estén colocados los datos inicialmente. Por tanto, cuadra perfectamente que los tiempos en las columnas de ordenado, inverso y aleatorio sean prácticamente idénticos.

EJERCICIO 3: algoritmo inserción

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de Inserción:

n	t ordenado	t inverso	t aleatorio
10000	0	682	338
20000	0	2714	1370
40000	0	10869	5436
80000	1	43354	21683
160000	3	174148	86085

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad del algoritmo de Inserción en sus casos medio y peor es $O(n^2)$. Al duplicar el tamaño del vector de entrada (), el número de operaciones debe multiplicarse por $2^2 = 4$. Por tanto, cuadra, ya que al dividir los tiempos de las columnas "inverso" y "aleatorio" de un valor de entre el anterior, el resultado está siempre en torno a 4.

En cuanto a los diferentes tiempos para un mismo valor de $O(n)$, también concuerdan con lo esperado, ya que la Inserción es extremadamente sensible al orden inicial. Por tanto, cuadra perfectamente que el caso ordenado registre tiempos casi nulos (baja su complejidad a al no tener que hacer ningún desplazamiento), el caso inverso sea el más lento (es el peor caso y hace todos los desplazamientos posibles) y el aleatorio se quede en un tiempo intermedio, tardando aproximadamente la mitad que el inverso.

EJERCICIO 4: algoritmo rápido

SE LE PIDE: Tras medir los tiempos, rellenar la tabla con el algoritmo de QuickSort:

n	$t \text{ ordenado}$	$t \text{ inverso}$	$t \text{ aleatorio}$
10000	3	3	3
20000	5	6	8
40000	11	13	17
80000	23	25	36
160000	48	53	82

Razone si los diferentes tiempos y su cociente concuerda con lo esperado.

Sí, los datos obtenidos concuerdan con lo esperado ya que la complejidad teórica del método Rápido (caso medio y mejor) es $O(n \log n)$. Al duplicar el tamaño del vector de entrada ($2n$), el tiempo de ejecución no se cuadruplica, sino que crece multiplicándose por un factor ligeramente superior a 2. Por tanto, cuadra, ya que al dividir el tiempo de un valor de entre el anterior en la tabla, el resultado se sitúa de forma constante en torno a 2.1 o 2.2, confirmando su eficiencia logarítmica.

En cuanto a los diferentes tiempos para un mismo valor de n , también concuerdan con la implementación del algoritmo. Al escoger el pivote en la posición central del vector (como hace el código de la práctica), se evita caer en el peor caso $O(n^2)$. Por tanto, cuadra perfectamente que los tiempos sean bajísimos en las tres columnas y que el caso aleatorio sea ligeramente más lento que el ordenado o inverso, ya que los vectores desordenados obligan al algoritmo a realizar un mayor número de intercambios (swaps) en la memoria.

EJERCICIO 5: algoritmo rápidoBajos vs inserciónBajos

TABLA 5= RÁPIDO vs. INSERCIÓN PARA TIEMPOS BAJOS
(tiempos en milisegundos y SIN_OPTIMIZACIÓN y para caso ALEATORIO)

<i>n</i>	<i>t rapido</i>	<i>t insercion</i>	<i>t rapido/t insercion</i>
10	127	149	0,852348993
15	211	239	0,882845188
20	287	307	0,93485342
25	420	399	1,052631579
30	503	517	0,972920696
35	625	586	1,066552901
40	695	705	0,985815603
45	793	845	0,938461538
50	926	962	0,962577963
55	1025	1175	0,872340426
60	1157	1291	0,896204493
65	1271	1312	0,96875
70	1393	1461	0,953456537
75	1534	1567	0,978940651
80	1716	1747	0,982255295
85	1800	1796	1,002227171
90	1941	1932	1,004658385
95	2035	2036	0,999508841
100	2210	2202	1,003633061

Con la tabla solicitada no se obtiene un resultado muy claro, pero para poder responder a la pregunta se hacen pruebas con un mayor

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041\p2>java -Xint p2.InsercionTiemposBajos aleatorio 100000
n=200    Tiempo=4974    Repeticiones=100000
n=205    Tiempo=5136    Repeticiones=100000
n=210    Tiempo=5161    Repeticiones=100000
n=215    Tiempo=5370    Repeticiones=100000
n=220    Tiempo=5439    Repeticiones=100000
n=225    Tiempo=5575    Repeticiones=100000
n=230    Tiempo=5764    Repeticiones=100000
n=235    Tiempo=5895    Repeticiones=100000
n=240    Tiempo=6108    Repeticiones=100000
n=245    Tiempo=6195    Repeticiones=100000
n=250    Tiempo=6444    Repeticiones=100000
n=255    Tiempo=6483    Repeticiones=100000
n=260    Tiempo=6747    Repeticiones=100000
n=265    Tiempo=7015    Repeticiones=100000
n=270    Tiempo=7195    Repeticiones=100000
n=275    Tiempo=7086    Repeticiones=100000
n=280    Tiempo=7253    Repeticiones=100000
n=285    Tiempo=7425    Repeticiones=100000
```

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041\p2>java -Xint p2.RapidoTiemposBajos aleatorio 100000
n=200    Tiempo=4924    Repeticiones=100000
n=205    Tiempo=5039    Repeticiones=100000
n=210    Tiempo=5195    Repeticiones=100000
n=215    Tiempo=5340    Repeticiones=100000
n=220    Tiempo=5512    Repeticiones=100000
n=225    Tiempo=5595    Repeticiones=100000
n=230    Tiempo=5887    Repeticiones=100000
n=235    Tiempo=6007    Repeticiones=100000
n=240    Tiempo=6119    Repeticiones=100000
n=245    Tiempo=6256    Repeticiones=100000
n=250    Tiempo=6509    Repeticiones=100000
n=255    Tiempo=6929    Repeticiones=100000
n=260    Tiempo=7183    Repeticiones=100000
n=265    Tiempo=7280    Repeticiones=100000
```

Tras hacer estas pruebas ves que con valores tan bajos la diferencia es mínima y aproximando, el valor en el que puede notarse la diferencia es entre $n = 250$ y $n = 300$.

De cara a implementar el algoritmo opcional para tratar de mejorar los tiempos se estableció un umbral a partir del cual el método empieza a utilizar el algoritmo de inserción. Teóricamente debería mejorar el tiempo, pero, quizás por las características del ordenador o por algún error a la hora de la implementación el tiempo no mejora.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041\p2>java -Xint p2.RapidoTiempos2 aleatorio 1
n=10000  Tiempo=230      Repeticiones=1
n=20000  Tiempo=438      Repeticiones=1
n=40000  Tiempo=1860     Repeticiones=1
n=80000  Tiempo=6714     Repeticiones=1
n=160000 Tiempo=40238     Repeticiones=1
```

PRACTICA P3

DyV por SUSTRACCIÓN

SE LE PIDE:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

Sustracción 1 ($O(n)$) y Sustracción 2 ($O(n^2)$): Los tiempos reflejan sus complejidades lineal y cuadrática (multiplicándose por 2 y por 4 respectivamente al duplicar n). Sin embargo, la naturaleza de la recursión por sustracción hace que la profundidad de la pila de llamadas sea igual a n . Esto provoca que ambas clases aborten prematuramente (StackOverflow) para valores pequeños (en torno a $n = 8000$), impidiendo procesar entradas grandes.

Sustracción 3 ($O(2^n)$): Los tiempos confirman su naturaleza exponencial. El algoritmo se vuelve intratable casi de inmediato; con solo sumar 1 o 2 unidades al valor de n , el tiempo se multiplica por 2 de forma agresiva, haciendo imposible superar valores de $n = 40$ o $n = 50$ en tiempos de ejecución razonables.

Contestar:

¿para qué valor de n dejan de dar tiempo (abortan) las clases

Sustracción1 y Sustracción2? ¿por qué sucede eso?

Las clases Sustracción 1 y Sustracción 2 dejan de dar tiempos (abortan) normalmente cuando supera valores justo después de 8000.

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041\p3>java -Xint p3.Sustraccion2 1
n=1000**TIEMPO=12**nVeces=1
n=2000**TIEMPO=30**nVeces=1
n=4000**TIEMPO=126**nVeces=1
n=8000**TIEMPO=488**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
    at p3.Sustraccion2.rec2(Sustraccion2.java:10)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
    at p3.Sustraccion2.rec2(Sustraccion2.java:15)
```

```
C:\Users\Colás\Desktop\Repos_Uniovi\alg_PerezFernandezNicolasU0306041\p3>java -Xint p3.Sustraccion1 1
n=1000**TIEMPO=0**nVeces=1
n=2000**TIEMPO=2**nVeces=1
n=4000**TIEMPO=0**nVeces=1
n=8000**TIEMPO=0**nVeces=1
Exception in thread "main" java.lang.StackOverflowError
    at p3.Sustraccion1.rec1(Sustraccion1.java:9)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
    at p3.Sustraccion1.rec1(Sustraccion1.java:14)
```

Esto ocurre porque se desborda la pila y se lanza StackOverflowError. Al ser algoritmos recursivos de sustracción simple ($n-1$), cada llamada genera un nuevo marco en la pila de ejecución.

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80?
Razonar la respuesta.

El algoritmo en Sustracción 3 tiene una complejidad de $O(n^2)$ debido a que realiza dos llamadas recursivas por cada unidad que se resta a n.

- Razonamiento: El número total de llamadas es aproximadamente 2^n . Para $n = 80$, tenemos 2^{80} operaciones.
- Cálculo:
 1. Operaciones: $1,2089 \cdot 10^{24}$.
 2. En años serían aproximadamente 38.000.000 de años
- Conclusión: Es un algoritmo intratable para valores de superiores a 40-50.

Implementar una clase Sustracción4.java con una complejidad $O(n^3)$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=100, 200, 400, 800, ... (hasta FdT).

n	Sust4 t(ms)
100	254
200	510
300	773
400	1029
500	1279
600	1552
700	1801
800	fdt
900	fdt

Implementar una clase Sustracción5.java con una complejidad $O(3n/2)$ (con su pertinente toma de tiempos) y después rellenar una tabla en la que se muestre el tiempo (en milisg.) para n=30, 32, 34, 36, ... (hasta FdT).

n	Sust5 t(ms)
30	2246
32	2402
34	2554
36	2704
38	2873
40	2997
42	fdt
44	fdt
46	fdt
48	fdt
50	fdt

Calcular: ¿cuántos años tardaría en finalizar la ejecución Sustracción5 para n=80?
Razonar la respuesta.

En este caso la complejidad es de $O(3^{n/2})$, por tanto:

1. Con $n = 80$, $3^{80/2} = 3^{40}$ que es más o menos $1,2157 \cdot 10^{19}$
2. Por tanto, en años sería más o menos 385,5 años

DyV por división:

SE LE PIDE:

Tras analizar la complejidad de las tres clases anteriores, no se le pide hacer sus tablas de tiempos, pero sí razonar si los tiempos coinciden o no la complejidad temporal de cada algoritmo.

División 1 ($O(n)$) y División 3 ($O(n)$): Ambas clases demuestran un comportamiento puramente lineal. Al duplicar el tamaño de la entrada ($2n$), el tiempo de ejecución empírico se multiplica por un factor muy cercano a 2. Al ser recursividad por división, la profundidad de la pila es logarítmica, lo que explica por qué estas clases no abortan y son capaces de procesar valores gigantescos (hasta $n = 1.024.000.000$) en fracciones de segundo.

División 2 ($O(n \log n)$): Los tiempos observados concuerdan con la complejidad. Al duplicar el tamaño del problema, el tiempo no se cuadruplica (como pasaría en un $O(n^2)$), sino que crece multiplicándose por un factor ligeramente superior a 2 (aprox. 2.1 o 2.2). Esto permite procesar volúmenes masivos de datos manteniendo una eficiencia altísima, de nuevo, sin desbordar la pila del sistema gracias a la división del problema.

Implementar una clase Division4.java con una complejidad $O(n^2)$ (con $a < bk$) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para $n=1000, 2000, 4000, 8000, \dots$ (hasta FdT).

n	$Div4\ t(ms)$
1000	14
2000	42
4000	169
8000	643
16000	2593
32000	10343
64000	41095

Implementar una clase `Division5.java` con una complejidad $O(n^2)$ (con $a > bk$) y la pertinente toma de tiempos. Después rellenar una tabla en la que se muestre el tiempo (en milisg.) para $n=1000, 2000, 4000, 8000, \dots$ (hasta FdT).

n	$Div5\ t(ms)$
1000	31
2000	88
4000	345
8000	1344
16000	5403
32000	21794

Dos ejemplos básicos:

SE LE PIDE:

Tras analizar la complejidad de los diversos algoritmos que hay dentro de las dos clases, ejecutarlas y tras poner en una tabla los tiempos obtenidos, comparar la eficiencia de cada algoritmo.

Para `VectorSuma.java`:

Método	Algoritmo	t	Complejidad
Método 1 (sum1)	Iterativo	532ns	$O(n)$
Método 2 (sum2)	Recursivo (Sustracción)	1211ns	$O(n)$
Método 3 (sum3)	Recursivo (División)	2852ns	$O(n)$

La recursividad (Métodos 2 y 3) introduce penalizaciones graves de tiempo y recursos debido a la gestión de la pila de llamadas. Para un problema elemental como sumar o recorrer todos los elementos de un array, la solución iterativa siempre será la más eficiente en la práctica. El diseño Divide y Vencerás (Método 3) solo justifica su sobrecoste temporal cuando consigue bajar la complejidad teórica del problema (por ejemplo, pasando de un $O(n^2)$ a un $O(n \log n)$). Por tanto, en problemas a gran escala si estaría justificado el uso de DyV.

Para `Fibonacci.java`

Método	Algoritmo	Tiempo	Complejidad
Método 1	Iterativo	374 ns	$O(n)$
Método 2	Iterativo alternativo	539 ns	$O(n)$
Método 3	Ecuación matricial	462 ns	$O(n)$ u $O(\log n)$
Método 4	Recursivo puro (DyV)	3.335 ms	$O(2^n)$

Practica DyV

PRIMERA PARTE

Resolver el problema mediante un algoritmo trivial con una clase `p3p.PuntosTrivial.java`, que calcule la solución calculando la distancia entre todo par de puntos diferentes, para así saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados, para comprobar el buen funcionamiento del algoritmo implementado:

n	$t_{Trivial}$
1000	23
2000	95
4000	337
8000	1340
16000	5279
32000	21115
64000	77690

SEGUNDA PARTE

Resolver el problema mediante un algoritmo trivial con una clase `p3p.PuntosDyV.java`, que calcule la solución calculando la distancia entre todo par de puntos diferentes, para así saber al final el par de puntos buscado y su distancia mínima.

Ejecutar la clase metiendo como parámetro cada uno de los nombres de los ficheros aportados, para comprobar el buen funcionamiento del algoritmo implementado:

n	t_{DyV}
1000	1
2000	1
4000	4
8000	6
16000	14
32000	28
64000	56

Práctica P4

JAVA

Razone convenientemente la complejidad temporal del algoritmo implementado en ColoreoGrafo.java

La complejidad teórica en el peor de los casos es $O(V + E)$ siendo V los vértices del grafo y E las aristas ("edges"). Sin embargo, dado que los grafos generados para esta práctica (cuadrículas) tienen una densidad de aristas limitada (cada nodo tiene un máximo de unos 8 vecinos), el número de aristas es directamente proporcional al número de vértices $O(V)$. Por lo tanto, la complejidad asintótica práctica del algoritmo se comporta de forma lineal $O(n)$ respecto al tamaño del problema.

Razone: ¿siguen la complejidad calculada los tiempos de la tabla?

Sí, los tiempos obtenidos empíricamente en la tabla confirman de manera clara la complejidad teórica lineal que calculamos previamente. De hecho, si lo graficásemos veríamos una línea recta que crece de forma lineal.

<i>reps</i>	<i>n</i>	<i>t Coloreado.java (ms)</i>	<i>t Coloreado.py (ms)</i>
1000	4	2	2
	8	1	3
	16	5	8
	32	10	18
	64	9	42
	128	17	80
	256	26	178
	512	54	340
	1024	100	761
	2048	293	1555
	4096	658	3250
	8192	1368	6514
	16384	3194	14269
	32768	6218	32355
	65536	13533	64418

PYTHON

Razone convenientemente la complejidad temporal del algoritmo implementado en coloreo_grafo.py

La complejidad teórica en el peor de los casos es $O(V + E)$ siendo V los vértices del grafo y E las aristas ("edges"). Sin embargo, dado que los grafos generados para esta práctica (cuadrículas) tienen una densidad de aristas limitada (cada nodo tiene un máximo de unos 8 vecinos), el número de aristas es directamente proporcional al número de vértices $O(V)$. Por lo tanto, la complejidad asintótica práctica del algoritmo se comporta de forma lineal $O(n)$ respecto al tamaño del problema.

Razone: ¿siguen la complejidad calculada los tiempos de la tabla?

Sí, los tiempos obtenidos empíricamente en la tabla confirman de manera clara la complejidad teórica lineal que calculamos previamente. De hecho, si lo graficásemos veríamos una línea recta que crece de forma lineal.

Práctica P5

Descripción del problema

El problema del ferry consiste en distribuir un conjunto de vehículos entre los dos carriles (babor y estribor) de un barco, ambos de igual longitud, con el objetivo de **maximizar el número de vehículos embarcados**. Cada vehículo tiene una longitud determinada y los dos carriles del ferry tienen la misma capacidad máxima. El problema requiere decidir, para cada vehículo, si va en babor o en estribor (o no embarca), garantizando que la suma de longitudes de cada carril no supere la longitud del barco.

Se trata de un problema de optimización combinatoria que se resuelve de forma eficiente mediante **programación dinámica**, evitando la exploración exhaustiva de todas las combinaciones posibles (2^n en fuerza bruta) mediante la reutilización de subproblemas ya resueltos.

Definición del estado

SE LE PIDE: Definir el estado de la tabla de programación dinámica empleada en el algoritmo.

El estado se define mediante una matriz booleana **dp[i][l]**, donde:

dp[i][l] = true indica que es posible haber considerado los primeros i vehículos con exactamente l metros acumulados en el carril de **babor** y (**sumatorio[i] - l**) metros en el carril de **estribor**, cumpliendo que ambas longitudes no superan la capacidad del carril del barco.

En el código de **Ferry.java**, esta definición se implementa directamente:

```
private boolean[][] dp; // dp[i][l] = true si es alcanzable el estado (i vehículos, l metros en babor)
private int[] sumatorio; // suma acumulada de longitudes: sumatorio[i] = sum(vehiculo 1..i)
```

Los índices de la tabla son:

- $i \in [0, n]$: número de vehículos considerados hasta el momento (0 = ninguno).
- $l \in [0, longitudCarril]$: metros asignados al carril de babor.

El estado inicial es **dp[0][0] = true**: con cero vehículos y cero metros en babor, el estado siempre es alcanzable.

Relación de recurrencia

SE LE PIDE: Describir la relación de recurrencia que rellena la tabla de DP.

Para cada vehículo i (de 1 a n) y para cada longitud de babor l alcanzable en el estado anterior $dp[i-1][l] = \text{true}$, se contemplan dos opciones:

1. Meter el vehículo i en babor: si $l + \text{longitud}[i] \leq \text{longitudCarril} \rightarrow dp[i][l + \text{longitud}[i]] = \text{true}$
2. Meter el vehículo i en estribor: si $\text{sumatorio}[i] - l \leq \text{longitudCarril} \rightarrow dp[i][l] = \text{true}$

La implementación en Ferry.java recorre la tabla de abajo hacia arriba (bottom-up):

```
for (int i = 1; i <= vehicles.size(); i++) {  
    for (int l = boatLength; l >= 0; l--) {  
        if (!dp[i-1][l]) continue;  
        // Opción 1: babor  
        if (l + vehicles.get(i-1) <= boatLength)  
            dp[i][l + vehicles.get(i-1)] = true;  
        // Opción 2: estribor  
        if (sumatorio[i] - l <= boatLength)  
            dp[i][l] = true;  
    }  
}
```

Nótese que el bucle interior recorre l de mayor a menor para evitar que los estados recién marcados en la fila i contaminen la exploración de esa misma fila. El número máximo de vehículos se extrae buscando la fila más alta de la tabla con algún valor true.

Complejidad

SE LE PIDE: Razonar la complejidad temporal y espacial del algoritmo.

Complejidad temporal: El algoritmo rellena una tabla de dimensiones $(n+1) \times (L+1)$, donde n es el número de vehículos y L es la longitud del carril. Para cada celda se realizan un número constante de operaciones, por lo que la complejidad temporal es:

$$O(n \cdot L)$$

Este coste es pseudopolinomial: depende del valor numérico de L , no del número de bits para representarlo. Para los casos de prueba del enunciado ($L \leq 50$, $n \leq 25$), el coste es completamente manejable.

Complejidad espacial: La tabla dp ocupa $O(n \cdot L)$ celdas booleanas. Adicionalmente, el array de sumatorios ocupa $O(n)$. En total, el espacio es $O(n \cdot L)$.

Comparación con fuerza bruta: Una solución de fuerza bruta que explorase todas las asignaciones posibles tendría complejidad $O(2^n)$, haciendo inviable su uso para n grandes. La programación dinámica reduce drásticamente este coste a un producto que crece linealmente en cada dimensión.

Implementación

SE LE PIDE: Describir las clases implementadas y verificar el correcto funcionamiento del algoritmo contra los casos de prueba.

El proyecto Maven en **p5/alg_ferry/Algoritmo_Ferry/** contiene las siguientes clases:

Clase	Responsabilidad
Ferry.java	Clase principal. Implementa el algoritmo DP (<code>algoritmoFerry()</code>), el cálculo de sumas prefijas, la extracción del máximo de vehículos y la reconstrucción del camino (<code>printPossibleAssignment()</code>).
App.java	Punto de entrada. Carga el fichero de prueba (por defecto <code>test02.txt</code>) mediante <code>LectorFicheros</code> y ejecuta <code>ferry.run()</code> .
LectorFicheros.java	Utilería de lectura. Parsea ficheros de texto con formato: primera línea = longitud del carril, segunda línea = longitudes de vehículos separadas por espacios. Devuelve un objeto <code>DatosFerry</code> .
FerryTests.java	Suite JUnit con 8 tests (<code>test00</code> a <code>test07</code>). Cada test carga su fichero, construye el <code>Ferry</code> , llama a <code>run()</code> y verifica con <code>assertEquals()</code> que el número máximo de vehículos coincide con el esperado en <code>resultados.txt</code> .

Un detalle de diseño relevante es el uso del tipo **record Step** (introducido en Java 16) para registrar cada paso de la asignación durante la reconstrucción del camino. El record almacena el estado anterior, el estado actual, el número de vehículo y el carril destino ("babor" o "estribor"), lo que hace el código de reconstrucción muy legible.

Conclusión

El algoritmo de programación dinámica resuelve el problema del ferry de forma óptima y eficiente. La clave está en la definición del estado `dp[i][l]` que condensa en una sola tabla todas las posibles combinaciones de asignación, y en la relación de recurrencia que distingue entre colocar el vehículo en babor o estribor. La complejidad $O(n \cdot L)$ es pseudopolinomial y completamente manejable para los tamaños de instancia del enunciado.

La implementación en `Ferry.java` refleja fielmente esta estructura: el bucle doble de relleno, el precálculo de sumas prefijas para calcular la carga de estribor en $O(1)$, y la reconstrucción recursiva del camino con el tipo `record Step`. Los 8 tests JUnit verifican el correcto funcionamiento sobre todos los casos de prueba aportados.

Práctica P6

TIEMPOS DEL ALGORITMO DE LOS CONTENEDORES

Complejidad del algoritmo

El algoritmo resuelve el problema de distribuir **n objetos** en el menor número posible de contenedores de capacidad **C** mediante backtracking.

Complejidad temporal: $O(n!)$ en el peor caso. Para cada objeto se prueba insertarlo en cada uno de los contenedores ya abiertos o en uno nuevo, lo que en el peor caso genera un árbol de decisión cuyo número de nodos crece factorialmente con n .

Complejidad espacial: $O(n^2)$, ya que la pila de recursión tiene profundidad n y en cada nivel se pueden almacenar hasta n contenedores.

El rendimiento real mejora considerablemente gracias a dos optimizaciones clave: la **ordenación descendente** del conjunto de objetos, que coloca primero los más grandes y permite encontrar buenas soluciones rápidamente, y la **poda por cota** (`contenedores.size() >= mejorK`), que descarta cualquier rama del árbol que ya haya superado el número de contenedores de la mejor solución encontrada hasta el momento. Ambas estrategias reducen drásticamente el espacio de búsqueda en la práctica.

<i>reps</i>	<i>Archivo</i>	<i>t (ms)</i>
100000	test00.txt	271
	test01.txt	188
	test02.txt	253
	test03.txt	192
	test04.txt	392
	test05.txt	456
	test06.txt	222
	test07.txt	245
	test08.txt	fdt
	test09.txt	fdt

Práctica P7

Descripción del problema y punto de partida

Esta práctica retoma el problema de **empaquetado en contenedores (bin packing)** trabajado en P6: dado un conjunto de objetos con sus tamaños y una capacidad de contenedor, encontrar el número mínimo de contenedores necesarios para almacenarlos todos. El algoritmo de P6 lo resolvía mediante **backtracking puro** con una poda simple (cortar si el número de contenedores abiertos ya iguala la mejor solución conocida). En P7 se mejora la poda mediante una **cota inferior (lower bound)** que permite descartar ramas de forma más anticipada y agresiva.

Los casos de prueba son **exactamente los mismos 10 ficheros** (test00.txt a test09.txt) que en P6, lo que permite una comparación directa de tiempos entre ambos algoritmos.

Heurístico mejorado: poda por cota inferior

SE LE PIDE: Describir el heurístico aplicado y cómo mejora la poda respecto al **backtracking puro** de P6.

La mejora principal de **AlmacenajeContenedoresRyP.java** respecto a **AlmacenajeContenedores.java** de P6 radica en el cálculo de una **cota inferior (lower bound)** al comienzo de cada llamada recursiva. Esta cota estima el número mínimo de contenedores adicionales que se necesitarán para los objetos aún no asignados, considerando el espacio libre que ya existe en los contenedores abiertos:

```
// 1. Suma de los tamaños de los objetos pendientes de asignar
int sumaRestante = 0;
for (int i = indexObject; i < conjuntoS.length; i++)
    sumaRestante += conjuntoS[i];

// 2. Espacio libre total en los contenedores ya abiertos
int espacioLibre = 0;
for (int i = 0; i < numContenedoresUsados; i++)
    espacioLibre += capacidadC - contenedoresActuales[i];

// 3. Volumen que no cabe en el espacio libre disponible
int necesario = Math.max(0, sumaRestante - espacioLibre);

// 4. Mínimo de contenedores nuevos requeridos (techo de la división)
int lowerBound = (necesario + capacidadC - 1) / capacidadC;

// 5. Poda: si incluso en el mejor caso posible no se mejora, se abandona la rama
if (numContenedoresUsados + lowerBound >= mejorK) return;
```

La fórmula del **lowerBound** es el **techo de (necesario / capacidadC)**, implementado en Java como **(necesario + capacidadC - 1) / capacidadC** para evitar importar la función `Math.ceil()`. Este valor indica cuántos contenedores vacíos serían necesarios en el mejor caso posible para los objetos restantes. Si incluso en ese escenario óptimo el total de contenedores no mejora la solución conocida, la rama se abandona sin explorarla.

Nótese que la condición de poda usa **>= mejorK** (no **> mejorK**): al ser una minimización, solo tiene sentido continuar si se puede estrictamente mejorar la solución actual, es decir, usar menos contenedores que **mejorK**.

Diferencias respecto a P6

A continuación se comparan las decisiones de diseño más relevantes entre ambas implementaciones:

Condición de poda	P6 — Backtracking puro	P7 — Backtracking + Lower Bound
Poda 1: contenedores abiertos $\geq$ mejorK	<code>contenedores.size() >= mejorK</code> → corta la rama	<code>numContenedoresUsados + lowerBound >= mejorK</code> → corta la rama
Poda 2: apertura de nuevo contenedor	No hay poda adicional; abre uno nuevo salvo que <code>size() >= mejorK</code>	<code>numContenedoresUsados + 1 < mejorK</code> → solo abre si tiene sentido
Cota inferior (lower bound)	No existe. La única cota es la propia mejorK.	<code>lowerBound = ceil((sumaRestante - espacioLibre) / capacidadC)</code> Estima el mínimo de contenedores extra necesarios.
Efecto sobre el árbol	Poda tarde: solo cuando ya se ha superado mejorK	Poda anticipada: descarta ramas antes de explorarlas basándose en el volumen restante.

Otra diferencia estructural relevante es que **AlmacenajeContenedoresRyP** utiliza **campos estáticos** en lugar de instancias de objeto (como en P6). Esto elimina la necesidad de pasar la lista de contenedores como parámetro en cada llamada recursiva y también elimina la copia profunda que hacía **copiar()** en P6; en su lugar, se usa **System.arraycopy()** sobre el array de asignaciones cuando se encuentra una solución mejor.

Complejidad y análisis práctico

Razone si los tiempos obtenidos concuerdan con la complejidad teórica esperada.

El problema del bin packing es **NP-difícil**, por lo que cualquier algoritmo exacto tiene complejidad exponencial en el peor caso. El backtracking sin poda explora un árbol con hasta **$O(n^n)$** nodos (cada objeto puede ir a cualquiera de los n contenedores posibles). Las podas reducen drásticamente los nodos visitados en la práctica, pero no cambian el peor caso teórico.

La poda de P6 (size $\geq$ mejorK) es una poda de cotas superiores: corta cuando ya se ha superado la mejor solución. La poda de P7 añade una **cota inferior basada en el volumen**: anticipa que incluso en el mejor escenario la rama no puede mejorar la solución. Esto permite cortar ramas mucho más temprano en el árbol, especialmente cuando los objetos son de tamaños similares o la capacidad es ajustada.

reps	Archivo	t (ms)
200000	test00.txt	82
	test01.txt	90
	test02.txt	102
	test03.txt	93
	test04.txt	142
	test05.txt	209
	test06.txt	103
	test07.txt	109
	test08.txt	346
	test09.txt	780

Conclusión

La práctica P7 demuestra cómo una poda más informada puede transformar el rendimiento de un algoritmo de backtracking sin alterar su corrección. La cota inferior implementada en **AlmacenajeContenedoresRyP** es **admisibile**: nunca sobreestima el número de contenedores necesarios, por lo que nunca descarta soluciones óptimas. Al mismo tiempo, es suficientemente ajustada como para eliminar grandes porciones del árbol de búsqueda, especialmente en instancias con muchos objetos.

La fórmula **lowerBound = ceil((sumaRestante - espacioLibre) / capacidadC)** captura con precisión el volumen de objetos que no cabe en el espacio ya disponible y calcula el mínimo de contenedores nuevos necesarios para acogerlos. Esta información, combinada con la condición **numContenedoresUsados + lowerBound $\geq$ mejorK**, permite podar en el momento en que cualquier solución descendiente de ese nodo no puede superar la solución ya conocida.

Los tiempos de la tabla comparativa (una vez rellenados) deberían confirmar esta mejora, siendo especialmente notable en test08 y test09 donde el espacio de búsqueda es mayor.